Quarry: Data Model and API Reference

Manoj Abhiram

November 10, 2026

CONTENTS

1. Conventions 3

2. Database 3

2.1 Storage pattern 3

2.2 Tables by domain 4

Identity and workspace 4

Lead generation 5

Jobs 5

Outreach 6

Enrichment, watch and integrations 6

Billing 7

Compliance and usage 7

2.3 Referential and deletion rules 7

3. Backend module map 8

4. Pipeline internals 9

4.1 LLM calls and failure handling 9

4.2 Scoring 9

4.3 Outreach logic 10

5. API catalogue 10

5.1 Authentication and account 11

5.2 Lead requests 12

5.3 Leads 13

5.4 Lists 14

5.5 Outreach 14

5.6 Watch (signals and saved searches) 15

5.7 Integrations 15

5.8 Workspaces and billing 16

5.9 Compliance and operations 17

6. Configuration reference 18

Document	Low-level design: database tables, module map, endpoint catalogue, conventions
Version	1.0
Date	2026-11-10
Source of truth	The current code. When this document and the code disagree, the code wins; the interactive reference is the Swagger UI at `/docs` on a running API

1. Conventions

- Base path: every API route is under `/api` (`API_V1_STR`). The public API for customers is under `/api/public/v1`.
- Format: JSON in, JSON out. Errors use FastAPI's `{"detail": ...}` shape; the global handler returns a generic message for unhandled 500s so internals do not leak.
- Auth: browsers send cookies and, on unsafe methods, `X-CSRF-Token`. API clients send `Authorization: Bearer <token>`. Public API keys (`qk_...`) are for `/public/v1` only.
- Workspace: when a user belongs to organizations, the active one is chosen with the `X-Org-Id` header; role checks then apply (`owner > admin > member > viewer`).
- Long operations return **202** with an id (`request_id` or `job_id`); poll status or follow the SSE stream.
- IDs: opaque strings (UUIDs). Authorization never relies on an ID being unguessable.

2. Database

2.1 Storage pattern

Every table has `id` (string primary key) and `data` (JSON document). Only fields used for filtering, sorting or uniqueness are real columns. Repositories (`app/db/*.py`) wrap each table in a `DocTable`. Engine settings: SQLite in WAL mode, `foreign_keys=ON`, one process-wide write lock.

2.2 Tables by domain

IDENTITY AND WORKSPACE

Table	Key columns	Notes
`users`	`email` (unique), `google_id` (unique), `org_id`	Encrypted Serper key and verifier key live in `data`; bcrypt password hash
`user_sessions`	`user_id`, `token_hash` (unique), `expires_at`	Hashed refresh tokens; expired rows purged at startup
`organizations`	`domain` (unique)	
`org_members`	`org_id`, `user_id`, `role`	Role is `owner`, `admin`, `member` or `viewer`
`org_invites`	`org_id`, `email`, `token_hash`, `status`	Invite token stored hashed; email sent in the background
`resource_shares`	`org_id`, `resource_type`, `resource_id`	Shares a request or list with the workspace

LEAD GENERATION

Table	Key columns	Notes
`requests`	`user_id`, `org_id`, `status`, `created_at`, `updated_at`	`failure_reason`, `queries`, list marks in `data`. Status is server-controlled: `Pending`, `Completed`, `Failed`
`product_intake`, `icp_profiles`	`request_id` (unique)	The user's product description and the ICP
`leads`	`request_id`, `user_id`, `url`, `match_score`, `enrichment_status`	Unique `(request_id, url)`. `enrichment_failed`, `enrichment_failure_reason`, email status, intent, `rejected_at` in `data`
`lead_enrichments`	`lead_id` (unique), `company_id`	Tech stack, infrastructure, signals, firmographics
`decision_makers`	`lead_id`	Name, title, email, LinkedIn URL, email status
`companies`	`canonical_domain`, `crunchbase_id`, `linkedin_id` (all unique)	Shared across requests, de-duplicated by domain; holds company facts only
`competitor_analyses`, `competitors`	`request_id` / `analysis_id`, `url`, `match_score`	
`lead_lists`, `lead_list_items`	`user_id`, `created_at`; `(list_id, lead_id)`	
`serper_cache`	`query` (unique), `expires_at`	Search result cache

JOBS

Table	Key columns	Notes
`jobs`	`kind`, `owner_id`, `status`, `run_at`, `created_at`	`payload`, `attempts`, `max_attempts`, `result`, `error`, `started_at`, `finished_at` in `data`. Statuses `queued`, `running`, `succeeded`, `failed`, `cancelled`

OUTREACH

Table	Key columns	Notes
`outreach_mailboxes`	`user_id`	SMTP and IMAP settings; passwords stored as `*_enc` (Fernet); `daily_cap`, `warmup`, `timezone`, `paused`, `pause_reason`, `imap_last_uid`
`outreach_sequences`	`user_id`, `status`	Up to 8 steps; statuses draft, active, paused
`outreach_enrollments`	`sequence_id`, `user_id`, `status`, `email`, `next_send_at`	One contact in one sequence. Terminal states: replied, bounced, unsubscribed, completed, suppressed, failed
`outreach_messages`	`enrollment_id`, `mailbox_id`, `message_id`, `direction`, `created_at`	Direction covers sent, reply, bounce; `Message-ID` used for threading

ENRICHMENT, WATCH AND INTEGRATIONS

Table	Key columns	Notes
`provider_keys`	`user_id`, `provider`	Encrypted Hunter, Snov and PDL keys
`enrichment_credit_ledger`	`user_id`, `created_at`	Email-lookup credits (separate from request credits)
`fallback_suppressions`	`owner_id`, `value`	Providers/values to skip in the waterfall
`saved_searches`	`user_id`	Filters, alert settings, `seen_ids`
`signal_events`	`user_id`, `domain`, `kind`, `detected_at`	Hiring, funding, news, tech change
`watch_snapshots`	`snapshot_key`	Last-seen state used to detect change
`api_keys`	`user_id`, `key_hash`	SHA-256 hash only; prefix kept for display
`webhooks`, `webhook_deliveries`, `webhook_subscriptions`	`user_id`; `webhook_id`, `status`, `next_attempt_at`; `scope_id`, `event`	Delivery log with attempts and next attempt time
`crm_connections`, `crm_sync`	`scope_id`, `provider`; `connection_id`, `lead_id`	Per-lead sync status

BILLING

Table	Key columns	Notes
`subscriptions`	`scope_id`, `provider_subscription_id`	Scope is the user or the organization
`credit_ledger`	`scope_id`, `kind`, `ref`, `created_at`	Append-only: grant, spend, top-up. Balance is the sum
`billing_events`	`event_key` (unique)	Idempotency record for payment webhooks
`invoices`	`scope_id`, `invoice_key`	

COMPLIANCE AND USAGE

Table	Key columns	Notes
`suppressions`	`owner_id`, `value`, `kind`	An email address or a whole domain; checked before enrichment, export and send
`audit_log`	`owner_id`, `action`, `created_at`	Append-only; holds no raw personal data
`usage_events`	`user_id`, `kind`, `request_id`, `cost_usd`, `created_at`	Calls and estimated cost per metered run
`schema_migrations`	`version`	Applied migration versions

2.3 Referential and deletion rules

- Deleting a **request** deletes its leads, decision makers and enrichments. Shared `companies` rows (facts about organizations) are kept.
- Deleting an **account** deletes the user's requests, leads, lists, sessions and personal data.
- **Erasure** by person removes the person from decision makers and leads across the workspace and writes an audit entry without the raw value.
- **Retention** purges rejected leads and un-promoted search results older than 30 days.

3. Backend module map

Module	Responsibility
`app/main.py`	App factory and lifespan: connect DB, run migrations, register job handlers, start the worker and scheduler, sweep orphaned requests; CORS, global error handler, routers
`app/jobs.py`	Durable queue and worker: `enqueue`, `register_handler`, `cancel_job`, `recover_running_jobs`
`app/core/`	`config`, `cookies` (sessions and CSRF), `url_guard` (SSRF), `rate_limit`, `security_crypto`, `scoring_config`, `logger`
`app/db/`	`engine`, `docstore`, `models`, `models_trust`, `growth_models`, `migrations`, one repository per table
`app/api/`	Routers: auth, google_auth, users, organizations, teams, requests, request_intake, leads, lists, competitors, tech_stack, jobs, verification, compliance, usage, billing, integrations
`app/services/`	Pipeline (`leads.py`, `pipeline_jobs.py`), LLM (`llm.py`), `scraper`, `serper`, `scoring`, tech-stack fingerprinting, company enrichment, `product_similarity`, `competitor_service`, `export_service`, `auth`, `email`
`app/outreach/`	`service` (mailboxes, sequences, send loop), `inbound` (IMAP polling, DSN), `logic` (templates, windows, warmup, signed tokens), `dnscheck`, `api`
`app/verification/`	`service`, `local` checks, `smtp` probe, `providers`, `types`
`app/waterfall/`	Enrichment `providers` and `service`, `claude_scoring`, `api`
`app/watch/`	`searches`, `signals`, `ports`, `api`
`app/integrations/`	Public API keys, outbound webhooks, `api`
`app/crm/`	`CrmConnector` interface, `hubspot`, `mapping`, `service`, `jobs`
`app/billing/`	`plans`, `service`, `store`, payment `providers` (Stripe, Razorpay, mock)
`app/teams/`	Role helpers, workspace dependencies, invite mailer
`app/compliance/`	`optout`, `person_erasure`, `retention`, `region`, `exports`, `audit`
`app/p3/`	Wiring for scheduled jobs and settings for outreach, enrichment and watch
`app/usage.py`, `provenance.py`, `suppression.py`	Metering and quotas, per-field provenance, suppression matching

4. Pipeline internals

4.1 LLM calls and failure handling

`_llm_json_completion(model, system_prompt, user_prompt, ..., raise_on_error=False, purpose="")`:

1. JSON-mode completion through LiteLLM with fallback models; if it errors or returns nothing, retry once without JSON mode.
2. Unwrap and parse. Failures are classified as `provider_error`, `empty_response`, `parse_error` or `not_an_object`.
3. The helper logs the failure (model, purpose, reason). With `raise_on_error=True` it raises `LLMCompletionError`; otherwise it returns `{}`.

`extract_lead_from_page` uses `raise_on_error=True`. On failure it keeps the lead with the deterministic scraped fields, sets `enrichment_failed`, a user-facing `enrichment_failure_reason` and a code. `POST /leads/{id}/retry-enrichment` enqueues `lead.retry_enrichment`, which re-runs extraction and overwrites the lead.

4.2 Scoring

Score	How it is computed
Match score (0 to 100)	From the LLM during extraction, stored on the lead
Intent score	Four pillars with caps: firmographic 25 (match score scaled), technographic 20, intent signals 35, engagement 20. Each pillar weight is multiplied by a confidence factor (1.0, 0.75 or 0.5). Intent and engagement decay 15% per 30 days since the last signal
Bands	HOT, WARM, NURTURE, WATCH, COLD, defined in `core/scoring_config.py`

The optional ICP filter (`ICP_FILTER_ENABLED`, default false; `ICP_SCORE_THRESHOLD`, default 60) drops low-scoring leads and employee sizes outside the ICP before saving. It never drops `enrichment_failed` leads.

4.3 Outreach logic

Concern	Rule
Templates	`{{variable}}` or `{{variable | fallback}}`; known variables are `first_name`, `last_name`, `name`, `title`, `company`, `email`, `domain`, `sender_name`; unknown variables are rejected and missing values reported in the preview
Sequence size	At most 8 steps
Send attempts	3 attempts; permanent SMTP errors fail at once
Window	Default 09:00 to 17:00 in the contact's timezone; sends outside it move to the next slot
Cap	`daily_cap` per mailbox, default 40, counted from local midnight
Warmup	`ceil(5 * 1.3 ^ days_since_start)` capped at `daily_cap`
Auto-pause	Hard-bounce rate above 2% once at least 20 messages are sent (env-tunable)
Headers	`List-Unsubscribe` (URL and mailto) and `List-Unsubscribe-Post: List-Unsubscribe=One-Click`
Footer	Unsubscribe link and sender postal address (`OUTREACH_POSTAL_ADDRESS`; a placeholder is used and flagged if unset)
Tokens	Unsubscribe and tracking tokens are signed (HMAC) and purpose-bound

5. API catalogue

Notation: **Auth** is `S` session or bearer, `K` public API key, `P` public (no login, rate limited), `W` provider-signed webhook. **Role** is the minimum workspace role when an organization is active.

5.1 Authentication and account

Method	Path	Auth	Purpose
POST	`/auth/register`, `/auth/login`	P	Create account / sign in; sets three cookies
POST	`/auth/refresh`	cookie or body	Rotate the refresh token (cookie use needs CSRF)
POST	`/auth/logout`	S	Delete session, clear cookies
POST	`/auth/forgot-password`, `/auth/reset-password`	P	Email-link reset
GET	`/auth/csrf`	S	Return the CSRF value for cross-site deployments
GET	`/auth/google`, `/auth/google/url`, `/auth/google/callback`	P	Server-side Google sign-in
GET / PUT / DELETE	`/users/me`	S	Profile; delete account and personal data
GET / PUT / DELETE	`/users/me/serper-key`	S	Own Serper key (encrypted, masked on read, validated on save)
POST, GET, PUT	`/organizations`, `/organizations/me`	S	Legacy single-organization endpoints

5.2 Lead requests

Method	Path	Purpose
GET / POST	`/requests`	List, create
PUT / DELETE	`/requests/{id}`	Update metadata (status is ignored); delete with cascade
PUT	`/requests/{id}/lists`	Set list marks
GET	`/requests/stats`	Dashboard statistics
POST	`/requests/analyze-source`, `/generate-icp`, `/check-consistency`, `/suggest-chip`	Wizard helpers
GET	`/requests/find-competitors`	Competitor suggestions
POST	`/requests/launch`, `/requests/launch-direct`	Start a discovery or direct-company run (202)
POST	`/requests/parse-companies-csv`	Parse an uploaded company list
POST	`/usage/estimate`, `/usage/sample-run`	Cost estimate; first search only, no commitment
GET	`/usage/summary`	Runs, calls, estimated cost and quota left this month

5.3 Leads

Method	Path	Purpose
GET	`/leads?request_id=`	Leads for a request
GET	`/leads/status/{request_id}` and `/stream`	Status by polling or server-sent events (header, `?token=` or cookie)
GET	`/leads/decision-makers`	People at the leads
POST	`/leads/{id}/retry-enrichment`	Queue a retry job
POST / GET	`/leads/intent/analyze`, `/leads/intent/{request_id}/{lead_index}`	Run and read per-lead intent analysis
POST	`/leads/verify-email`, `/leads/verify-emails-batch`, `/leads/{id}/verify-email`	Verify emails
POST	`/verification/decision-makers/{dm_id}/verify`	Verify one decision maker’s email
GET / PUT / DELETE	`/verification/settings`, `/verification/settings/key`	Verifier provider and key
POST	`/enrichment/leads/{lead_id}/waterfall`	Run the waterfall for a lead
GET	`/enrichment/credits`	Email-lookup credit balance
PUT / DELETE	`/enrichment/keys/{provider}`	Store or remove a provider key
POST / GET	`/enrichment/scoring/batch`, `/enrichment/scoring/batch/{id}`	Batch AI scoring
GET	`/leads/{request_id}/export/{companies|competitors|decision-makers}?format=csv|xlsx|pdf`	Export

5.4 Lists

Method	Path	Purpose
GET / POST	`/lists`	List, create
GET / PUT / DELETE	`/lists/{id}`	Read, rename, delete
POST / DELETE	`/lists/{id}/leads`	Add or remove leads (ownership checked on both the list and the leads)
GET	`/lists/{id}/export`	Export a list

5.5 Outreach

Method	Path	Purpose
GET / POST	`/outreach/mailboxes`	List, connect
POST	`/outreach/mailboxes/test`	Test SMTP and IMAP login before saving
PUT / DELETE	`/outreach/mailboxes/{id}`	Update, remove
POST	`/outreach/mailboxes/{id}/resume`, `/preflight`, `/poll`	Resume after pause; SPF/DKIM/DMARC check; poll now
GET / POST	`/outreach/sequences`	List, create
PUT / DELETE	`/outreach/sequences/{id}`	Update, delete
POST	`/outreach/sequences/{id}/preview`, `/start`, `/pause`, `/enroll`	Preview, start, pause, enroll contacts
GET	`/outreach/sequences/{id}/stats`, `/enrollments`	Sent, replied, bounced; contact states
GET / POST	`/public/unsubscribe/{token}`	Recipient unsubscribe (P)
GET	`/public/o/{token}.gif`	Optional open pixel (P)

5.6 Watch (signals and saved searches)

Method	Path	Purpose
GET / POST	`/watch/searches`	List, create
PUT / DELETE	`/watch/searches/{id}`	Update, delete
POST	`/watch/searches/{id}/run`	Run now
GET	`/watch/signals`, `/watch/leads/{lead_id}/signals`	Signal feed
POST	`/watch/scan`	Scan now

5.7 Integrations

Method	Path	Purpose
GET / POST / DELETE	`/integrations/api-keys`	Create (plaintext shown once) and revoke public API keys
GET / POST	`/integrations/webhooks`; PATCH / DELETE `/{id}`	Manage webhooks
POST / GET	`/integrations/webhooks/{id}/test`, `/deliveries`	Test event; delivery log
GET	`/integrations/export/{source}/{source_id}.csv`; POST `/integrations/export`	CSV for automation
PUT / GET / POST / DELETE	`/integrations/hubspot/token`, `/oauth/url`, `/oauth/callback`; `DELETE /integrations/hubspot` disconnects	HubSpot connection
GET / PUT	`/integrations/hubspot/mapping`	Field mapping
POST	`/integrations/hubspot/push`	Push a list; returns 202 and `job_id`
GET	`/integrations/hubspot/sync?list_id=`	Per-lead sync status and counts
GET	`/public/v1/requests`, `/requests/{id}/leads`, `/leads/{id}`, `/lists`, `/lists/{id}/leads`	Customer-facing read API (K)

Webhook events: `outreach.reply`, `outreach.bounce`, `outreach.unsubscribe`, `signal.detected`, `alert.saved_search`, and others defined in `integrations/webhooks.py`. Each delivery is signed: `X-Quarry-Signature: t=<unix>,v1=HMAC_SHA256(secret, "<t>." + body)`; receivers should reject timestamps older than 5 minutes.

5.8 Workspaces and billing

Method	Path	Role	Purpose
GET / POST	`/orgs`	member	List my workspaces; create one
POST	`/orgs/invites/accept`	S	Accept an invite token
GET / PUT	`/orgs/{id}`	member / admin	Workspace details
GET / PUT / DELETE	`/orgs/{id}/members[/{user_id}]`	admin	List, change role, remove
GET / POST / DELETE	`/orgs/{id}/invites[/{invite_id}]`	admin	Invites; the API returns the link at once and sends email in the background (`email_status`)
POST / DELETE	`/orgs/{id}/shares[/{type}/{resource_id}]`	member	Share or unshare a request or list
GET	`/orgs/{id}/shared/requests[/{id}]` , `/shared/lists[/{id}]`	viewer	Read teammates' shared items
GET	`/billing/plans` , `/billing/summary` , `/billing/invoices`	member	Plans, balances and quota, invoices
GET / POST	`/billing/topup`	admin	Credit packs; start checkout
POST	`/billing/checkout`	admin	Plan checkout
POST	`/billing/webhooks/{provider}`	W	Signed, idempotent provider callback

5.9 Compliance and operations

Method	Path	Purpose
GET / POST / DELETE	`/compliance/suppressions`	View, add, remove suppression entries
POST	`/compliance/opt-out-link`	Generate a signed opt-out link
POST	`/compliance/erase`	Erase a person across the workspace
GET	`/compliance/audit-log`	Compliance audit trail
POST	`/compliance/leads/{id}/reject`, `/restore`	Reject (30-day deletion clock) or restore
POST	`/compliance/retention/run`	Run the retention purge now
GET / POST	`/public/opt-out`	Public opt-out page backend (P, rate limited)
GET	`/jobs`, `/jobs/{id}`; POST `/jobs/{id}/cancel`	Caller’s jobs (payloads never returned)
GET	`/health`	Liveness for containers

6. Configuration reference

Variable	Required	Default	Purpose
`SECRET_KEY`	yes	none	JWT signing; also the root of the credential-encryption key
`SERPER_API_KEY`	yes	none	Server default Serper key (users may store their own)
`DATABASE_URL`	no	`sqlite+aiosqlite:///./quarry.db`	Database
`LITELLM_MODEL`, `LITELLM_MODEL_MINI`, `LITELLM_FALLBACK_MODELS`	no	OpenRouter / Groq models	Model routing
`GROQ_API_KEY`, `OPENROUTER_API_KEY`, `ANTHROPIC_API_KEY`	per provider	none	LLM provider keys
`ALLOWED_ORIGINS`, `FRONTEND_URL`, `PUBLIC_API_URL`	no	localhost	CORS and absolute links
`COOKIE_SECURE`, `COOKIE_SAMESITE`, `COOKIE_DOMAIN`	no	secure when https; `lax`	Session cookie attributes
`JOB_WORKER_CONCURRENCY`, `JOB_POLL_INTERVAL_SECONDS`, `JOB_MAX_ATTEMPTS`, `JOB_BACKOFF_*`	no	4, 1.0, 3	Worker tuning
`ICP_FILTER_ENABLED`, `ICP_SCORE_THRESHOLD`	no	false, 60	Optional score filter
`QUOTA_RUNS_PER_MONTH`, `QUOTA_COST_USD_PER_MONTH`	no	30, 20	Per-user usage quota
`SERPER_USD_PER_CALL`, `LLM_USD_PER_CALL`	no	0.001, 0.002	Cost estimate unit prices

Variable	Required	Default	Purpose
`P3_SCHEDULER`, `P3_SCHEDULER_INTERVAL`	no	true, 60	Periodic job scheduler
`OUTREACH_POSTAL_ADDRESS`	for sending	placeholder	Footer sender address
`OUTREACH_BOUNCE_PAUSE_THRESHOLD`, `OUTREACH_BOUNCE_MIN_SAMPLE`	no	0.02, 20	Mailbox auto-pause
`ENRICH_FREE_CREDITS`	no	100	Starting email-lookup credits
`CLAUDE_BATCH_SCORING`, `CLAUDE_SCORING_MODEL`	no	false, `claude-haiku-4-5`	Batch scoring
`PAYMENT_PROVIDER` and Stripe / Razorpay keys	for billing	`mock`	Payment provider selection

`.env.production.example` in the repository documents every variable the app reads.